Introduction to Embedded Systems

Course Code: 244066

Lab 4

STM32 Direct Memory Access (DMA) Interface

By

Dr. Muhammad Imran Nadeem

Zhengzhou University

Instructions

- Complete every task in the lab practice session.
- For each task, include a 2–5 line description along with a screenshot of both your code and its output.
- Submit your lab report by email to Imran.Nadeem@outlook.com, using the subject line format as:
- **ES24-[lab number]-[registration number]**.
- Lab report need to be submitted before **11:59 pm**.
- Submissions received after the deadline will not be accepted.

Overview

Overview of Learning Outcomes

By completing these simulated labs, you will:

- Understand DMA Operation
- Configure DMA Modes
- Work with DMA Registers
- Simulate and Analyze Results
- Apply DMA in Embedded Design

Setup Notes

- Use the same project template as the earlier labs: device **STM32F103C4**, **CMSIS Core** + **Device Startup**, #include "**stm32f10x.h**" (plus <**stdint.h**>), Debug tab set to **Use Simulator**, and the same **STMaccess.ini** (MAP 0x40000000,0x400FFFFF READ WRITE) so the **DMA1, RCC and USART1 registers** can be accessed.
- Set the **optimization** level to -O0 (Options for Target → C/C++) so the compiler does not remove your copy loops.
- Verify results in the **Watch window** (Hex format). For arrays, also use **View** → **Memory Windows** → **Memory 1** with the array name as the address.
- Keep buffers small: the **STM32F103C4** has only 6 KB of RAM.
- The simulator may not move data for the **DMA controller**. If the destination buffer does not change after you enable a channel, finish every **register-configuration** and **Watch-window** verification step, then model the controller by hand as each task describes. Use a small buffer (for example, 4 items) when modelling by hand.
- If a **while** loop waiting on a DMA flag hangs, set **DMA1->ISR** in the Watch window to 0x00000002 (transfer complete for channel 1).

Task 1: Enable the DMA1 Clock and Explore the Channel Registers

Steps

1. Create a new project and add a main.c file.
2. In main(), enable the DMA1 clock with RCC->AHBENR |= RCC_AHBENR_DMA1EN; (bit 0).
3. Start the debug session and add these to the Watch window: RCC->AHBENR, DMA1_Channel1->CCR, DMA1_Channel1->CNDTR, DMA1_Channel1->CPAR, DMA1_Channel1->CMAR and DMA1->ISR.
4. Verify that bit 0 of RCC->AHBENR is 1.
5. Leave the channel disabled (EN = 0). Write test values: CNDTR = 10, CPAR = 0x20000000, CMAR = 0x20000100. Read them back in the Watch window.
6. Make a four-row table that maps each register (CCR, CPAR, CMAR, CNDTR) to its role in the chapter: control register, source address, destination address, count. Add the courier analogy from the slides for each.
7. In 2–3 sentences, explain why the DMA clock must be enabled before the registers can be used, and why the DMA controller needs the AHB bus to reach memory and peripherals.
8. Note that these registers can only be changed while the channel is disabled.

Task 2: CPU-Driven Copy Baseline

Steps

1. Declare #define BUF_SIZE 32 and two global arrays, uint32_t src[BUF_SIZE] and uint32_t dst[BUF_SIZE].
2. In main(), fill src with the values 1 to BUF_SIZE using a loop. Leave dst at zero.
3. Copy the data with a CPU loop: for (i = 0; i < BUF_SIZE; i++) dst[i] = src[i];.
4. Start the debug session and open the Registers window. Find Internal → States and Sec. Add src and dst to the Watch window.
5. Set one breakpoint just before the copy loop and one just after. Record the States value at each and subtract them to get the cost of the CPU copy.
6. Confirm that dst matches src.
7. Change BUF_SIZE to 64 and record the cost again. Note how it scales with the amount of data.
8. In 2–3 sentences, explain why this method keeps the CPU busy and blocks other work, using the limitations from the chapter.

Task 3: Memory-to-Memory Transfer with DMA (Polling the Flag)

Steps

1. Reuse src and dst from Task 2 and comment out the CPU copy loop. Clear dst to zero.
2. Enable the DMA1 clock and make sure channel 1 is disabled: DMA1_Channel1->CCR &= ~0x1;.
3. Load the registers. In memory-to-memory mode, CPAR holds the source and CMAR holds the destination: DMA1_Channel1->CPAR = (uint32_t)src;, DMA1_Channel1->CMAR = (uint32_t)dst;, DMA1_Channel1->CNDTR = BUF_SIZE;.
4. Configure CCR: MEM2MEM (bit 14) = 1, DIR (bit 4) = 0, PINC (bit 6) = 1, MINC (bit 7) = 1, PSIZE and MSIZE = 32-bit (10), PL = medium (01). In numbers, write DMA1_Channel1->CCR = 0x00005AC0;.
5. Start the transfer by setting EN: DMA1_Channel1->CCR |= 0x1;. Verify that CCR now reads 0x00005AC1.
6. Wait for completion with while (!(DMA1->ISR & (1 << 1)));, then clear the flags with DMA1->IFCR = (1 << 0); (write 1 to clear).
7. Verify that dst matches src, CNDTR is 0, and the TCIF1 bit (bit 1) of DMA1->ISR was set.
8. Simulator check: if dst is still all zeros after EN was set, your simulator does not model DMA data movement. Set BUF_SIZE to 4, copy the four values by hand in the Watch window, set CNDTR to 0 and DMA1->ISR to 0x2, then continue. State in your report that you modelled the controller by hand.
9. If the simulator did perform the transfer, record the States cost as in Task 2 and compare it with the CPU copy.

Task 4: Address Increment and Data Size Experiments

Steps

1. Write a function DMA_M2M_Run(uint32_t ccr_flags, uint16_t count) that disables channel 1, loads CPAR = src, CMAR = dst, CNDTR = count, writes CCR = ccr_flags, sets EN, waits for TCIF1, and clears the flags.
2. Fill src with distinct values (for example, src[i] = 0x1000 + i). Clear dst before every case.
3. Before running anything, write your predicted contents of dst for each case in a table.
4. Case A (baseline): PINC = 1, MINC = 1, 32-bit, count = BUF_SIZE. Use CCR = 0x5AC1.
5. Case B: PINC = 0, MINC = 1, 32-bit. Use CCR = 0x5A81.
6. Case C: PINC = 1, MINC = 0, 32-bit. Use CCR = 0x5A41.
7. Case D: PINC = 1, MINC = 1, 8-bit sizes, count = 16. Use CCR = 0x50C1.
8. Run each case and compare the real dst with your prediction. Use the Memory 1 window to see the result.
9. In 3–4 sentences, explain what PINC and MINC control, which case acts as a fast “fill” of a memory area, and why Case C leaves most of dst untouched.
10. If the simulator does not model DMA, still verify the four CCR values in the Watch window and complete the prediction table with a written explanation.

Task 5: Peripheral-to-Memory: Circular DMA Reception on USART1 RX

Steps

1. Reuse USART1_Init() with TE, RE and UE enabled. Declare uint8_t rx_buf[16] = {0}; and volatile uint8_t head = 0;.
2. Enable the DMA1 clock. USART1 RX uses DMA1 channel 5.
3. Load channel 5: CPAR = (uint32_t)&USART1->DR;, CMAR = (uint32_t)rx_buf;, CNDTR = 16;.
4. Configure CCR: DIR = 0 (peripheral to memory), CIRC (bit 5) = 1, MINC = 1, PINC = 0, 8-bit sizes, PL = medium. Write CCR = 0x10A0, then set EN so it reads 0x10A1.
5. Set the DMAR bit (bit 6) in USART1->CR3. Verify that CR3 reads 0x40.
6. In the main loop, compute head = 16 - DMA1_Channel5->CNDTR;. Add rx_buf, head and DMA1_Channel5->CNDTR to the Watch window.
7. Send 20 characters (A to T) by typing them in the UART #1 window. Fallback: for each character, write it to rx_buf[head] and decrement CNDTR by hand. When CNDTR reaches 0, reload it to 16, which is what circular mode does.
8. Record rx_buf, head and CNDTR after 5, 16 and 20 characters. After 20 characters you should see head = 4, CNDTR = 12, and rx_buf[0..3] = Q R S T.
9. In 2–3 sentences, explain why circular mode suits continuous reception, and what happens to old data that the CPU has not read yet.